

TIME COMPLEXITY

Type 1: Single For Loop with Linear Increment/Decrement

1 Single For Loop

Consider the following type of loop:

```
1 for(int i = 0; i < n; i++)  
2 {  
3     // statements  
4 }
```

Here, the loop variable i changes by a constant amount in every iteration.
For example:

- $i = i + 1$
- $i = i + 2$
- $i = i + 5$
- $i = i + 10$
- $i = i - 2$

These loops have a **linear time complexity**.

2 Counting the Number of Iterations

To count the number of times a loop executes, we first identify:

- **Lower Limit:** The value from where i starts.
- **Upper Limit:** The value up to which i goes.

For a loop that changes by 1, we use:

$$\text{Number of iterations} = \sum_{i=l}^u 1$$

Using the counting formula:

$$\sum_{i=l}^u 1 = u - l + 1$$

where:

- l = lower limit, i.e., starting value of i

- u = upper limit, i.e., last value taken by i

Therefore:

$$\text{Number of iterations} = \text{Upper Limit} - \text{Lower Limit} + 1$$

3 Important Point: Identify the Upper Limit Carefully

The upper limit is the **last value actually taken by i** .

For example:

```
1 for(int i = 0; i < n; i++)
```

The condition is:

$$i < n$$

Therefore, the last value of i is from 0 to n :

$$n$$

Hence:

$$l = 0, \quad u = n$$

and:

$$\text{Iterations} = n - 0 + 1 = n + 1 = n$$

4 Example 1: $i = i + 1$

Consider:

```
1 for(int i = 0; i < n; i++)
2 {
3     cout << i;
4 }
```

Step 1: Identify Lower Limit

The loop starts with:

$$i = 0$$

Therefore:

$$l = 0$$

Step 2: Identify Upper Limit

The condition is:

$$i < n$$

Therefore, the last value taken by i is:

$$u = n$$

Step 3: Apply the Formula

$$\text{Number of iterations} = \sum_{i=0}^{n-1} 1$$

Using:

$$\sum_{i=l}^u 1 = u - l + 1$$

we get:

$$= n - 0 + 1$$

$$= n$$

Step 4: Big-O Complexity

$$T(n) = n$$

Therefore:

$$T(n) = O(n)$$

5 Example 2: $i = i + 1$ with Different Lower Limit

Consider:

```
1 for(int i = 1; i <= n; i++)
2 {
3     cout << i;
4 }
```

Step 1: Identify Lower Limit

The loop starts from:

$$i = 1$$

Therefore:

$$l = 1$$

Step 2: Identify Upper Limit

The condition is:

$$i \leq n$$

Therefore:

$$u = n$$

Step 3: Apply the Formula

$$\text{Number of iterations} = \sum_{i=1}^n 1$$

$$= u - l + 1$$

$$= n - 1 + 1$$

$$= n$$

Step 4: Big-O Complexity

$$T(n) = n$$

Therefore:

$$T(n) = O(n)$$

6 Example 3: Starting from a Constant

Consider:

```
1 for(int i = 5; i <= n; i++)  
2 {  
3     cout << i;  
4 }
```

Step 1: Lower Limit

The loop starts from:

$$l = 5$$

Step 2: Upper Limit

The condition is:

$$i \leq n$$

Therefore:

$$u = n$$

Step 3: Apply the Formula

$$\text{Number of iterations} = \sum_{i=5}^n 1$$

$$= u - l + 1$$

$$= n - 5 + 1$$

$$= n - 4$$

Step 4: Big-O Complexity

$$T(n) = n - 4$$

Ignoring the constant:

$$T(n) = O(n)$$

7 Example 4: Upper Limit $2n$

Consider:

```
1 for(int i = 10; i <= 2*n; i++)
2 {
3     cout << i;
4 }
```

Step 1: Lower Limit

The loop starts from:

$$l = 10$$

Step 2: Upper Limit

The condition is:

$$i \leq 2n$$

Therefore:

$$u = 2n$$

Step 3: Apply the Formula

$$\text{Number of iterations} = \sum_{i=10}^{2n} 1$$

$$= u - l + 1$$

$$= 2n - 10 + 1$$

$$= 2n - 9$$

Step 4: Big-O Complexity

$$T(n) = 2n - 9$$

Ignoring constants:

$$T(n) = O(n)$$

8 Quick Reference Table

Loop	Number of Iterations	Big-O
i=0; i<n; i++	n	$O(n)$
i=1; i<=n; i++	n	$O(n)$
i=5; i<=n; i++	$n - 4$	$O(n)$
i=10; i<=2n; i++	$2n - 9$	$O(n)$
i=0; i<=n; i=i+10	$\frac{n}{10} + 1$	$O(n)$
i=5; i<=n; i=i+5	$\frac{n}{5}$	$O(n)$
i=n; i>=0; i=i-2	$\frac{n}{2} + 1$	$O(n)$
i=n; i>=0; i=i-5	$\frac{n}{5} + 1$	$O(n)$

9 Easy Rule for Students

When you see a **single loop**, follow these steps:

1. Find where i starts $\rightarrow$ **Lower Limit**.
2. Find the last value taken by $i \rightarrow$ **Upper Limit**.
3. Find the step size.
4. Count the iterations.
5. Express the result using Big-O notation.

$$\boxed{\text{Iterations} = u - l + 1}$$

Finally, for all the above examples:

$$\boxed{T(n) = O(n)}$$